

Отладка системы фокусов в Wasaby

Общие вопросы отладки

Где сейчас фокус?

Смотрим в консоли `document.activeElement`. Будет выведен сфокусированный на сайте элемент. Такая программная проверка на местоположение фокуса работает точнее.

Для удобства отслеживания сфокусированного элемента можно использовать функцию [Live Expression](#) в DevTools.

Часто люди отлаживают в DevTools и удивляются, почему фокус не в поле ввода (не моргает каретка). Не моргает, потому что сейчас контекст работы — DevTools, а не сайт. Чтобы каретка снова начала моргать, можно просто кликнуть на таб браузера.

Как отследить события фокуса?

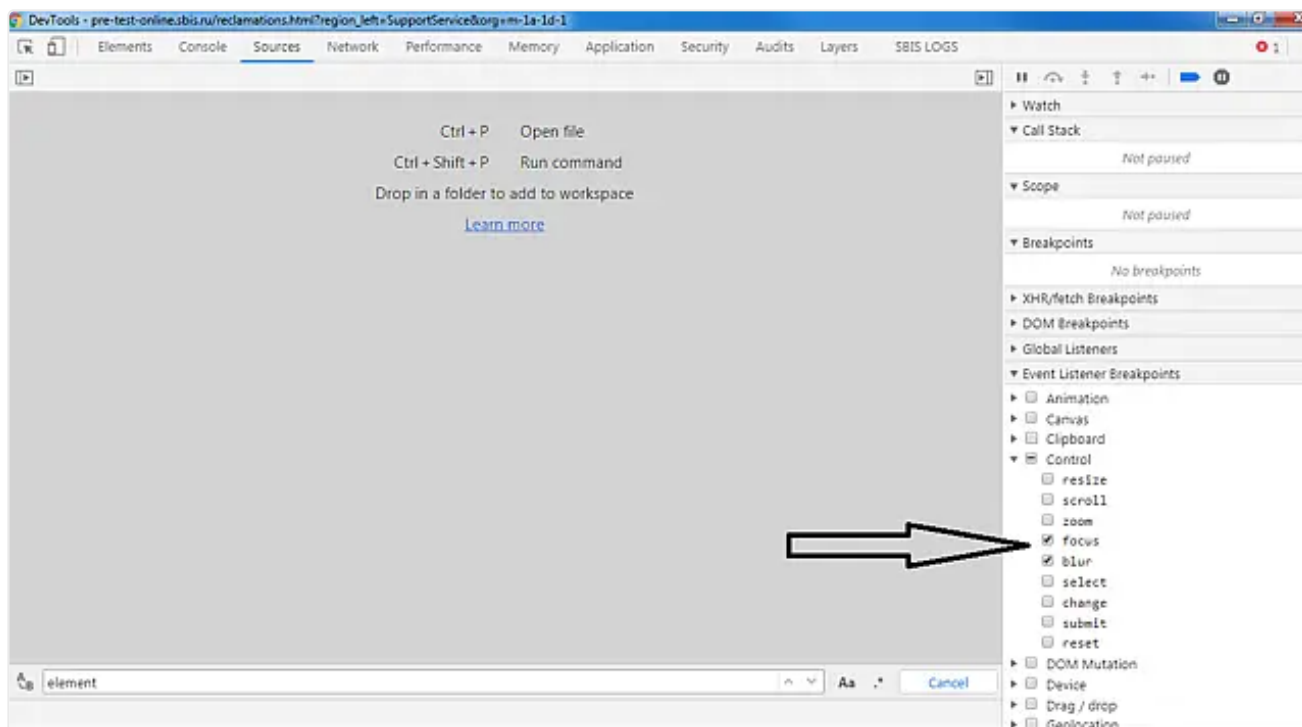
▼ Пример кода

```
1 document.body.addEventListener('focus', function(e) {  
2     console.trace(e.target);  
3 }, true);
```

На body подписываемся на этапе погружения события. Выводим `console.trace(e.target)`; или что-то еще. Таким образом можно отладить и порядок обхода по табу, и куда фокус переходит программно при вызове метода `activate()`, открытии и закрытии попапов и т.п.

Как отловить вызов фокуса?

Можно так:



Можно так:

```
1 window.myFocus = HTMLElement.prototype.focus;  
2 HTMLElement.prototype.focus = function() {  
3     console.trace();
```

```
4   return window.myFocus.apply(this, arguments);  
5 }
```

Этот способ удобнее в плане гибкости.

Можно попробовать отловить вызов фокусировки, поставив точки останова в метод `activate` и `UI/Focus:focus`.

Что система фокусов НЕ делает

Часто ошибки не связаны с системой фокусов, но любые проблемы с фокусами отправляются на отладку в платформу. Это неправильный подход. Если все ошибки по фокусам будут сразу лететь ответственным за систему фокусов, у них не останется времени ни на что кроме отладки ошибок.

Ошибки предварительно необходимо отлаживать по месту их проявления в соответствии с документацией по системе фокусов. Ошибка может быть на любом уровне контролов:

- на прикладных уровнях контролов
- на платформенном уровне контролов

Хорошим основанием полагать, что ошибка связана с системой фокусов, будет факт несоответствия документации реальному поведению.

Фокусировка на клик

Система фокусов не обрабатывает клик. При клике на элемент он фокусируется нативно. Причем на любом контроле по умолчанию стоит `tabindex="0"`, так что клик в область контрола сфокусирует его. Если на клик фокусы ведут себя как-то иначе, нужно искать причину в настройке контролов и их настройке. Может быть разработчик

- поставил `tabindex="-1"`;
- позвал `preventDefault` у события `click`, `mousedown`, `mouseup`;
- перевел фокус в другое место;
- использовал атрибут `ws-no-focus`.

все это не относится к системе фокусов и должно отлаживаться по месту.

Лишние вызовы `activate`

Система фокусов не зовёт `activate` нигде, кроме автофокусировок.

Если где-то в процессе работы фокус улетает не туда, значит так настроены эти контролы. `activate` и `UI/Focus:focus` (замена нативному фокусу) зовут разработчики. Можно найти место вызова фокусировки и спросить у разработчика, зачем он переводит фокус.

Восстановление фокуса

Система фокусов не восстанавливает фокус никак кроме одного описанного в документации способа. Это восстановление фокуса нужно, чтобы сохранить контекст фокуса для дальнейшего управления с клавиатуры. Там просто фокусируется ближайший доступный родительский контейнер.

Таким образом, неожиданного перемещения фокуса быть не может. Система фокусов не пытается угадать, где фокус должен оказаться — это настраивают пользователи, вызывая методы `activate` и `focus`.

Обход по табу

Отладить обход по табу удобно помощью [подписки на событие](#).

Отладить поиск элемента для фокусировки можно методом [findFirstInContext](#) — поиск первого элемента в текущем контексте `rootElement` (элементе, внутри которого нужно искать элемент)

Метод [findWithContexts](#) — поиск следующего элемента после текущего `fromElement` в текущем контексте `rootElement` (элементе, внутри которого нужно искать элемент)

В обходе по табу лишние элементы

Включение элемента в обход по табу означает, что элемент — фокусируемый. Определение фокусируемости в системе фокусов аналогично нативному определению. Подробнее об этом можно почитать в документации по HTML или [здесь](#).

В обходе по табу лишний контрол

Если контрол обходится по табу — значит внутри него есть что фокусировать. Чтобы исключить контрол из обхода — нужно задать `tabindex="-1"`.

Если табиндекс задан, возможно он не применился, потому что контролам нужно задавать атрибуты с использованием префикса [attr](#):

```
1 <!-- WML -->
2 <Controls.Button attr:tabindex="-1"/>
```

В обходе по табу пропустился элемент

Пропуск элемента означает, что элемент — нефокусируемый. Определение фокусируемости в системе фокусов аналогично нативному определению. Подробнее об этом можно почитать в документации по HTML или [здесь](#).

В обходе по табу пропустился контрол

Если контрол не обходится по табу — значит внутри него нечего фокусировать. Такой контрол не будет фокусироваться, пока не [определить в его контроле](#) или его дочерних контролах элемент, который сможет сфокусироваться.

Неправильная последовательность обхода

Необходимо [настроить правильную последовательность](#) обхода с помощью табиндексов.

Активация

Важно понимать, что метод [activate](#) работает с готовой версткой на странице. Какую верстку построил шаблонизатор и какую отрисовал VDOM, вот такая верстка и будет использоваться при поиске элемента для фокусировки.

Для локализации ошибки необходимо проверять, что верстка удовлетворяет ожидаемому виду, прежде чем начинать отлаживать сам метод.

В случае вопросов к методу, его можно отладить.

Отладка метода `UI/Focus:focus` и событий описана [здесь](#).

При активации фокус уходит не туда

Фокус уходит туда, куда находит путь согласно настройкам атрибутов `ws-autofocus` и `tabindex`.

При этом `ws-autofocus` должен использоваться только для автофокусировки при открытии новой области, такой как попап или весь сайт. Возможно кто-то переусердствовал и поставил его в лишнее место, чтобы решить свою проблему своей активации, не учитывая, что активация может звать и из других мест.

Возможно неправильно настроены табиндексы и активация находит тот элемент, который имеет наименьший табиндекс в шаблоне, а затем [начинает искать элемент в другой ветке](#) HTML-структуры. Это касается и обхода по табу.

При активации фокус остается на месте

Если фокус остался на месте, вероятно, метод `activate` вообще не нашел элемента для фокусировки. В таком случае метод вернет `false` в качестве результата.

Либо метод `activate` нашел тот же самый элемент, который уже был в фокусе. В таком случае метод вернет `true` в качестве результата.

После попытки сфокусироваться есть проверка, что фокусировка прошла успешно. В консоль выведется ошибка, если фокусировался скрытый элемент.

Чтобы узнать, что именно работает не так и почему элемент не находится, нужно отладить метод `activate`.

События `activated`, `deactivated`

Отладка событий описана [здесь](#).

На клик не срабатывают события

Может быть фокус изначально был не там, где предполагалось. Если фокус переходит между дочерними контролами, у родительского контрола [события срабатывать не будут](#).

Возможно, события не срабатывают, потому что фокус не переходит. Фокус может не переходить, потому что на элементе (или на одном из его родителей), в который мы кликаем, стоит атрибут `ws-no-focus="true"`, который блокирует перевод фокуса.

Событие происходит на контроле несколько раз подряд

Сейчас такое поведение обусловлено особенностями работы системы событий. Обработчик на событие случается несколько раз, если контрол обернут в [НОС](#) или сам является НОС.

Проблемы с подскроллами

Отладка метода `UI/Focus:focus` описана [здесь](#).

Лишние подскроллы к фокусируемому элементу

Причины подскролла могут быть разные.

1. Это может быть программный вызов подскролла, можно отловить событие `scroll`.
2. Подскролл к фокусируемому элементу — это нативное поведение браузера. Так что подскролл может происходить из-за использования нативного метода `focus`. Вместо нативного `focus` можно использовать `UI/Focus:focus` с опцией `enableScrollToElement: false`.

Нет подскролла к фокусируемому элементу

Нужно звать `UI/Focus:focus` с опцией `enableScrollToElement: true`.

Экранная клавиатура

Экранная клавиатура показывается, когда на мобильных устройствах фокусируется поле ввода. То есть если кликнуть в поле ввода или сфокусировать его программно, часть экрана займет клавиатура для ввода текста.

Клавиатура показывается, а не должна

Если экранная клавиатура показывается, значит фокусируется поле ввода.

Если не нужно показывать экранную клавиатуру, не нужно фокусировать поле ввода.

Нужно понять, откуда зовется фокусировка и решить проблему по месту вызова фокусировки. За фокусировку отвечает разработчик контрола.

Вариант не фокусировать поле ввода — звать `activate` с опцией `enableScreenKeyboard = false`. Будет сфокусировано не поле ввода, а его контейнер.

Клавиатура не показывается, а должна

Есть очевидные варианты: клавиатура не показывается, потому что не фокусируется поле ввода. Возможно фокусируется какой-то другой элемент, или фокусировка не происходит вообще. Нужно ответить на вопрос: откуда зовется (должна зваться) фокусировка. Если фокусировка вообще не зовется — почему она не зовется? Если фокусируется не тот элемент — почему? За фокусировку отвечает разработчик контрола.

Менее очевидный вариант. На iPad есть баг, есть фокусировка поля ввода происходит программно, и в стеке вызова есть асинхронная фаза и источник вызова — не обработчик клика, экранная клавиатура не покажется. Она покажется позже при следующем клике в любую другую область. iPad как будто бы откладывает отображение экранной клавиатуры до следующей синхронной фокусировки.

Мы не нашли решения этой проблемы. Поэтому мы позволяем этой ситуации произойти. Например, при автофокусировке зовется `activate` с опцией `enableScreenKeyboard = false`, таким образом если при активации будет найден элемент поля ввода, фокусироваться будет не он, а его контейнер. Таким образом поле ввода не будет фокусироваться и баг с экранной клавиатурой не повторится.

Чтобы экранная клавиатура показывалась, нужно синхронно сфокусировать поле ввода методом `UI/Focus:focus` или `activate`, передав опцию `enableScreenKeyboard = true`.

Попапы

Открылся попап, по Esc не закрылся

Попап закрывается, если реагирует на событие нажатия клавиши Esc. События всплывают от сфокусированного элемента. Если попап не закрылся, фокус находится не в попапе и клавиша Esc всплывала из другого места. После открытия попапа нужно посмотреть [где находится фокус](#). Если фокус не попал в попап, нужно разобраться — что пошло не так.

Возможно попап настроен так, что автофокусировка после открытия вообще не зовется. Нужно изучить API опций попапа и убедиться, что он настроен правильно. Если он настроен правильно, после открытия будет вызываться метод `activate`.

Возможно автофокусировка зовется, но метод `activate` не смог ничего найти в попапе, что мог бы сфокусировать, в том числе он не смог сфокусировать и сам контейнер корневого контроля, расположенного внутри попапа. Подробнее можно [посмотреть тут](#).

Открылся попап, фокус не в поле ввода

Если после открытия фокус не там, где предполагалось, сначала можно проверить, фокусируется ли попап вообще. После открытия он должен закрываться по Esc. Подробнее можно посмотреть в [предыдущем пункте](#).

Если в попапе фокусируется не тот элемент, значит нужно настроить `ws-autofocus` и `tabindex`. Подробнее можно [посмотреть тут](#).

Закрылся попап, фокус не там

При закрытии попапа элемент, который был в фокусе, пропадает и фокус не знает куда ему деваться. В этом случае фокус "слетает" в `body`. Чтобы избежать этого, в системе фокусов есть [восстановление фокуса](#). Фокус будет восстановлен на ближайший доступный контейнер среди родителей контроля, потерявшего фокус. Причем родители определяются с учетом [опенеров](#). А надо понимать, что все попапы связаны между собой по опенерам. То есть попап, открывающийся из другого попапа, будет иметь в качестве опенера тот попап.

Проблема может возникнуть, если пользователь решил сам настроить опцию опенер, и не смог сделать это правильно. Например, передал в качестве опенера не то, что надо.

Скорее всего, после закрытия был сфокусирован контейнер согласно восстановлению фокуса по умолчанию, а пользователь хотел, чтобы фокус перешел в другой элемент. Тогда пользователь, ответственный за контрол, должен после закрытия попапа сам сфокусировать тот элемент, который хочет, потому что только он знает, какой должен быть сфокусирован элемент. Для этого он может воспользоваться методом `activate` или `UI/Focus:focus`.

Автофокусировка

Загрузилась страница, фокус попал не туда

Нужно [отладить метод](#) `activate`.

Переключил вкладку, фокус попал не туда

Нужно [отладить метод](#) `activate`.

Открыл попап, фокус попал не туда

Нужно [отладить метод](#) `activate`.

Проблемы с iPad

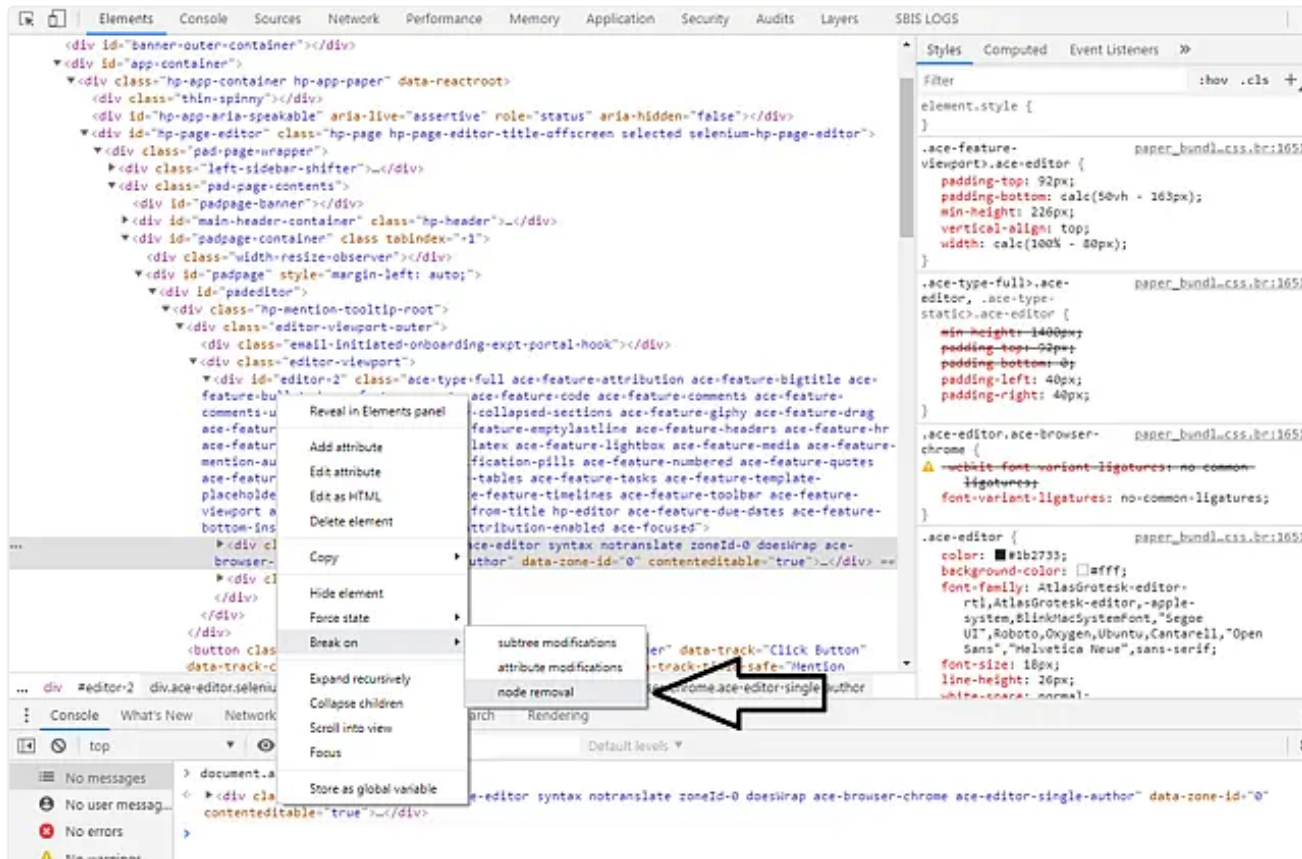
Поле ввода не фокусируется

У iPad есть баг. Подробно описано [здесь](#).

Фокус переместился при перерисовке

Предварительно нужно проверить [известные способы](#) перемещения фокуса — вдруг кто-то специально перемещает фокус. Если после перерисовки средствами VDOM сфокусированный элемент потерял фокус, а элемент (или аналогичный ему) при этом остался в DOM, это платформенная ошибка синхронизатора VDOM. Ее нужно передать платформе. Но нужно убедиться, что потеря фокуса произошла именно по вине перерисовки, а не по вине контроля. Для этого нужно отловить момент удаления сфокусированного элемента из DOM.

Потеря фокуса всегда связана с удалением элемента (или его родителя) из DOM. Настраиваем на нужном элементе Break on node removal.



Смотрим по стеку, что является причиной удаления элемента. Если в стеке есть код, который явно удаляет какую-то область в шаблоне с помощью изменения свойства, или явно удаляет/скрывает область сам, значит это проблема контроля (может быть прикладного, может быть платформенного). Если по стеку видно, что происходит просто перерисовка данных, или какие-то несущественные изменения, не связанные с изменением видимости областей — это может быть проблема VDOM.

Ошибки в консоли, связанные с системой фокусов

Ошибка в консоли "Плохой параметр fromElement"

fromElement — элемент, после которого ищется элемент для фокусировки.

В систему фокусов в метод поиска элемента был передан неправильный параметр. По сути поиск фокусируемого элемента был запущен для элементов, которых нет в DOM. Либо переданный элемент не является элементом.

Нужно ставить точку останова в место ошибки, и смотреть по стеку, откуда и кем передается неправильный элемент.

[См. ссылку](#)

Ошибка в консоли "Плохой параметр rootElement"

rootElement — родительский элемент, в контексте которого ищется элемент для фокусировки.

В систему фокусов в метод поиска элемента был передан неправильный параметр. По сути поиск фокусируемого элемента был запущен для элементов, которых нет в DOM. Либо переданный элемент не является элементом.

Нужно ставить точку останова в место ошибки, и смотреть по стеку, откуда и кем передается неправильный элемент.

[См. ссылку](#)

